

Day05 Part B 学习文档 v1.0 : LLM 如何决定调用 Tool (Tool Decision Layer)

本文是《从零实现 Agent Runtime》学习阶段的 Day05 Part B 正式学习文档。

Day05 Part A 已经建立了 Tool Calling 的基础模型：LLM 不直接执行外部动作，而是生成 Tool Call Intent，Runtime 负责验证、执行并把 Observation 写回 Runtime State。Part B 继续向前追问一步：在 Tool Call Intent 产生之前，LLM 为什么会决定调用 Tool，而不是直接回答？

本节定位

Day04 回答的是：

Runtime 如何让 LLM 更好地思考？

核心是：

Context Engineering

Day05 Part A 回答的是：

Runtime 如何让 LLM 的思考产生行动？

核心是：

Tool Calling

Day05 Part B 回答的是：

LLM 如何在“直接回答”和“调用 Tool”之间选择下一步行动？

核心是：

Tool Decision

本节的核心结论是：

Tool Decision 不是简单的函数匹配，而是 Goal-driven Action Selection：Agent 先理解目标，再基于当前 Runtime State、Available Tools 和 Runtime Policy 选择下一步行动。

目录

与 Part A 的联系

一个核心问题：LLM 为什么会调用 Tool

Tool Decision 的本质：Action Selection

Tool Decision 不是 if/else Rule

Tool Schema 如何影响决策

Tool Choice 的三种模式

Goal-driven Action Selection

Tool Decision 的三因素模型

为什么 Agent 有时候不调用 Tool

模型能力对 Tool Decision 的影响

Decision Layer 与 Execution Layer 分离

Tool Decision 的不确定性

Runtime 控制权分配问题

mini-agent-runtime 中的最小数据模型

工业级实现 Notes

工业术语映射

面试视角

写书素材

本 Part 核心认知升级

下一节学习计划

本章思考题

与 Part A 的联系

Part A 完成的是 Tool Calling 的基础定位。

以前的 ChatBot 链路是：

```
LLM
|
v
回答用户
```

Part A 把它升级为：

```
LLM
|
| 产生 Tool Call Intent
v
Runtime
|
| 验证 + 执行
v
Tool
|
| Observation
v
Runtime State
|
v
下一轮 Context
```

Part A 解决的是：

Tool Calling 是什么，以及它在 Runtime Loop 中的位置。

Part B 解决的是：

LLM 为什么知道什么时候应该调用 Tool？

这意味着 Part B 站在 Tool Call Intent 产生之前，研究的是 Agent 的 Decision Layer。

一个核心问题：LLM 为什么会调用 Tool

例如用户说：

查询一下我的订单现在在哪里。

Runtime 给 LLM 提供一个 Tool：

```
{
  "tools": [
    {
      "name": "query_order",
      "description": "查询订单状态",
      "parameters": {
        "order_id": "string"
      }
    }
  ]
}
```

为什么 LLM 不直接回答：

您的订单已经发货。

而是输出：

```
{
  "tool": "query_order",
  "arguments": {
    "order_id": "123"
  }
}
```

原因不是 LLM 突然拥有了真实 API 调用能力。

更准确地说：

Tool Definition 改变了 LLM 的决策空间。

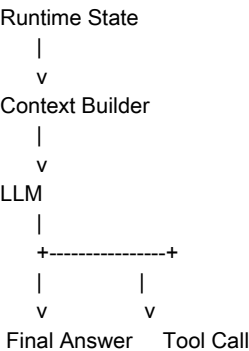
Runtime 把工具描述放进上下文之后，LLM 看到的不只是“可以回答文本”，还看到了“可以选择某种外部行动”。于是模型的下一步输出空间从单一文本生成，扩展为：

Final Answer
Tool Call

这就是 Tool Decision 的起点。

Tool Decision 的本质：Action Selection

从 Agent Loop 看，LLM 每一轮并不是只做“回答生成”，而是在选择下一步动作。



在这一层，LLM 面临的问题是：

现在应该：

- A. 直接回答？
- B. 调用工具？
- C. 调用哪个工具？
- D. 参数是什么？

这就是：

Action Selection

工业里也常见这些名字：

- Tool Selection
- Action Selection
- Function Selection
- Capability Routing
- Decision Layer

所以 Tool Decision 不是“调用函数”的工程细节，而是 Agent Runtime 中“下一步行动选择”的核心环节。

Tool Decision 不是 if/else Rule

很多初学者会把 Tool Calling 理解成：

```
if (user.includes("订单")) {  
  callOrderTool();  
}
```

这不是 Agent，而是 Rule Engine。

传统 Bot 更像：

```
用户输入  
|  
v  
关键词匹配  
|  
v  
固定流程  
|  
v  
接口调用
```

Agent 更像：

```
用户输入  
|  
v  
Context  
|  
v  
LLM reasoning  
|  
v  
Action decision  
|  
v  
Tool call
```

两者对比：

| 维度 | 传统 Bot | Agent | | --- | --- | --- | | 决策者 | 规则 | LLM + Runtime | | 决策方式 | if/else | 基于上下文的概率推理 | | 扩展方式 | 增加规则 | 增加能力与约束 | | 泛化能力 | 弱 | 强 |

Agent 的关键不是没有规则，而是 Runtime 不把所有行为都写死在 if/else 中，而是把可用能力暴露给 LLM，让 LLM 在受控空间里提出下一步行动。

Tool Schema 如何影响决策

这里要连接 Day04 的 Context Engineering。

Day04 学的是：

Context 决定 LLM 看什么。

Day05 进一步学习：

Tool Schema 决定 LLM 能做什么。

例如一个模糊的 Tool：

```
{
  "name": "search",
  "description": "搜索文件"
}
```

模型只能理解：

我可以搜索文件。

如果换成：

```
{
  "name": "search_workspace_files",
  "description": "Search source code, documents and configuration files inside current workspace"
}
```

模型会更容易理解：

这是当前工作区内搜索代码、文档和配置文件的能力。

所以 Tool Schema 本质上是在给 LLM 描述行动空间。

它会影响：

- Tool 选择
- 参数生成
- 调用频率
- 行为边界
- 错误恢复

因此 Tool Description 不只是普通 Prompt Engineering，更准确地说属于：

Tool Schema Engineering

进一步属于：

Agent Behavior Engineering

Claude Code 这类 Coding Agent 之所以强，不只是模型强，也因为 Runtime 暴露给模型的能力边界很清晰：

```
ReadFile
Search
EditFile
RunCommand
```

GitDiff

如果只暴露一个万能工具：

```
execute_workspace_action()
```

模型反而更难稳定判断什么时候用、怎么用、用到什么边界为止。

Tool Choice 的三种模式

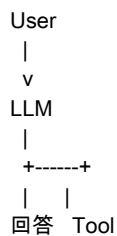
很多 Agent Runtime 都有类似 Tool Choice 的控制参数。

auto

默认模式：

LLM 自己决定

流程：



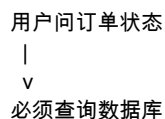
适合普通聊天 Agent 或泛化任务。

required

强制调用 Tool：

必须选择 Tool

例如订单客服：



这类场景不能让模型凭经验猜：

我猜您的订单应该到了。

工业系统中，这通常用于关键数据访问、合规查询、计费、订单、库存等需要真实外部状态的场景。

none

禁止 Tool：

只允许回答

例如用户要求：

只总结当前对话，不访问外部系统。

或者 Runtime 判断当前阶段不允许外部动作时，可以把 Tool Choice 设置为 none。

这三个模式说明：

Tool Decision 并不完全属于 LLM，Runtime 也可以控制模型的行动空间。

Goal-driven Action Selection

学习记录中有一个重要纠偏：

Tool Decision 不应该被割裂成“模型选择函数”。

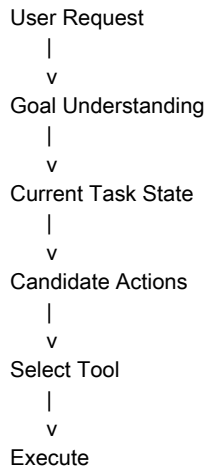
它应该放回我们 Day01 到 Day04 反复建立的链路：

Goal -> Plan -> Action

所以 Part B 的更准确表述是：

Tool Decision = Goal-driven Action Selection

完整链路：



也就是说，LLM 不是直接从“用户问题”跳到“调用某个 Tool”。

它应该先理解：

我要解决什么问题？

然后再判断：

为了推进这个目标，下一步应该做什么？

例如用户说：

帮我分析这个项目为什么启动失败。

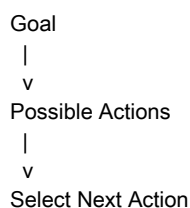
Goal 是：

Find root cause of startup failure

候选 Action 可能是：

- A. 直接回答
- B. 读取 package.json
- C. 查看错误日志
- D. 运行 npm install
- E. 运行测试或启动命令

此时 Decision Layer 做的是：



这也解释了为什么 Agent Loop 不是每一轮都重新决定目标。

Goal 是长期目标：

Improve function performance

Action 是短期一步行为：

ReadFile -> EditFile -> RunTest -> GitDiff

Runtime State 负责保持 Goal 和 Progress：

```
{
  goal: "optimize function",
  progress: [
    "read file",
    "found O(n^2) loop"
  ],
  nextAction: "edit"
}
```

所以 Agent 的状态演化更像：

Goal 保持稳定
Action 随 Observation 不断变化

Tool Decision 的三因素模型

Tool Decision 不是单因素决定。

正式学习记录里把它抽象为：

Tool Decision =
Context
+
Tool Definition
+
Runtime Policy

Context

来自 Day04：

用户目标
历史消息
Memory
Runtime State
Observation

它决定：

当前问题是什么？
当前任务进展到哪里？
下一步缺什么信息或动作？

Tool Definition

来自 Day05 Part C：

name
description
parameters

它决定：

有哪些能力可以被选择？
每个能力适合什么场景？
参数应该如何生成？

Runtime Policy

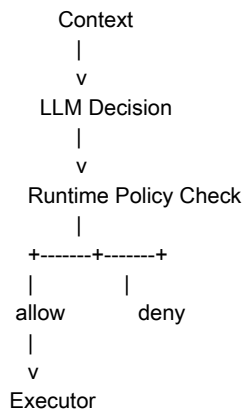
来自后续 Part F :

- permission
- guardrail
- human approval
- cost limit
- security boundary

它决定 :

- 模型提出的行动是否被允许 ?
- 是否需要人工确认 ?
- 是否需要重试、降级或阻止 ?

完整链路 :



这也是为什么工业 Agent 一定不是 :

LLM + Tools

而是 :

LLM
+
Runtime Control Layer
+
Tools

为什么 Agent 有时候不调用 Tool

Tool 存在不代表一定会调用。

例如用户问 :

什么是 React ?

同时 Runtime 提供 :

- search_web
- database_query
- calculator

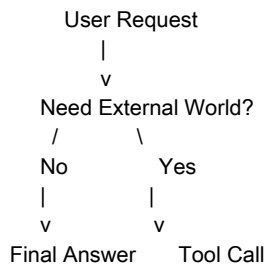
LLM 可能仍然直接回答。

原因是模型会评估 :

- 已有知识是否足够 ?
- 是否需要外部信息 ?
- 是否需要外部动作 ?
- 是否需要验证 ?

如果已有知识足够，合理输出就是 Final Answer，而不是 Tool Call。

决策图：



这点很关键。

很多企业 Agent 会犯的错误是：

给 LLM 一堆工具，然后希望它自动变聪明。

但工具越多，Decision Space 越大。Tool 设计不好，Agent 不但不会更强，反而可能退化为误调用、不调用、乱生成参数或频繁请求人工确认。

模型能力对 Tool Decision 的影响

学习记录里还有一个重要问题：

如果 LLM 不够聪明，即便给它 tools，它是不是也可能不知道调用，然后直接输出？

回答是：方向正确，但需要更准确地说：

不是“LLM 不知道 Tool 存在”，而是 LLM 的 Tool Decision 能力不足时，即使 Runtime 提供了 Tool，它也可能无法正确判断什么时候应该用 Tool。

例如 Runtime 提供：

```
{
  "name": "weather_query",
  "description": "查询城市天气"
}
```

用户问：

上海今天热吗？

优秀模型会判断：

这是实时天气问题
我不知道当前天气
存在 weather_query 工具
应该调用

输出：

```
{
  "tool": "weather_query",
  "city": "上海"
}
```

能力较弱或上下文理解不足的模型可能直接回答：

上海夏天通常比较炎热，建议注意防晒。

问题在于，它没有意识到：

这是一个需要实时数据的问题。

所以 Tool Decision Quality 取决于：

Model Capability
+
Tool Schema Design
+
Context Quality
+
Runtime Policy

不要简单理解为“模型越大越好”。更关键的是：

- 模型是否训练过 tool-use / function-calling 数据；
- Tool Schema 是否边界清晰；
- Context 是否明确表达目标、状态和可用能力；
- Runtime 是否能验证、约束和纠正不可靠决策。

这也是 Claude Code 等 Coding Agent 给人“像高级工程师”的原因：

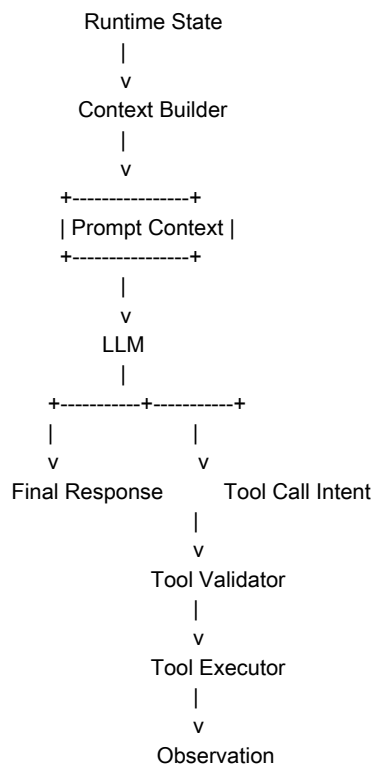
Claude Model
+
工程 Tool
+
优秀 Tool Schema
+
Context Engineering
+
Runtime Loop

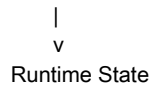
Decision Layer 与 Execution Layer 分离

Part B 必须强调一件事：

LLM 输出的是决策结果，不是执行结果。

更完整的 Runtime 流程：





LLM 不知道 Tool 是否真实存在，也不知道：

- API 是否成功；
- 权限是否允许；
- 数据是否存在；
- 调用是否超时；
- 操作是否收费；
- 行动是否会破坏用户数据。

LLM 只知道 Runtime 提供给它的能力描述。

因此必须区分：

Decision Layer
Execution Layer

LLM 负责：

我要做什么？
我建议调用哪个工具？
参数是什么？

Runtime 负责：

是否允许？
参数是否合法？
是否需要确认？
如何执行？
执行结果如何进入状态？

Tool Decision 的不确定性

Tool Decision 不是二值判断：

```
if need_tool == true
```

而更接近一个概率选择过程：

Given:

Context
+
Available Tools
+
User Intent

↓

Probability Distribution

↓

Select Next Action

例如用户问：

北京今天多少度？

Runtime 提供：

```
weather(city)
search(query)
```

模型内部的候选动作可能接近：

```
weather:    0.92
search:     0.05
final_answer: 0.03
```

最后选择：

```
weather
```

如果用户问：

服务器是不是挂了？

候选动作可能更不确定：

```
check_server_status: 0.55
answer_directly:    0.45
```

这时 Runtime 可以介入：

```
if (confidence < threshold) {
  requireConfirmation();
}
```

或者：

```
低置信度
|
v
重新规划
|
v
询问用户
```

这会连接后续章节：

- Reflection
- Self Correction
- Human Approval
- Guardrail
- Runtime Policy

所以 Agent Decision 本质上更像：

```
Next Action Prediction
```

而不是：

```
Function Matching
```

Runtime 控制权分配问题

Part B 最后补充了一个隐藏但重要的认知点：

Tool Decision 本质是 Runtime 的控制权分配问题。

很多人把 Agent 理解成：

LLM + Tools = Agent

但工业 Runtime 真正关心的是：

谁拥有下一步行动的决定权？

模式 1：完全由 LLM 决定

User

|

v

LLM

决定：

- 是否调用 Tool
- 调哪个 Tool
- 参数是什么

|

v

Runtime 执行

优点是灵活。

缺点是不可控。

模式 2：完全由 Workflow 决定

例如客服退款：

用户退款

|

v

固定流程

|

v

查询订单

|

v

检查规则

|

v

创建退款单

优点是稳定。

缺点是泛化弱。

模式 3：工业 Agent 主流模式

更多生产 Agent 是混合控制：

Goal

|

v

LLM Decision

|

v

Runtime Policy

|

v

Tool Call

|

v

Workflow Constraint

|

v

Execution

也就是说：

LLM 提出行动建议，Runtime 决定是否允许这个行动发生。

这解释了为什么企业 Agent 通常不是：

LLM + Tools

而是：

LLM
+
Workflow
+
Tools
+
Runtime Control

mini-agent-runtime 中的最小数据模型

Part B 之后，mini-agent-runtime 至少需要一个决策层数据结构：

```
type AgentDecisionType = "tool_call" | "final_answer";
```

```
interface AgentDecision {  
  type: AgentDecisionType;  
  finalAnswer?: string;  
  toolCall?: ToolCallIntent;  
}
```

```
interface ToolCallIntent {  
  name: string;  
  arguments: Record<string, unknown>;  
}
```

LLM 可能输出：

```
{  
  "type": "tool_call",  
  "toolCall": {  
    "name": "search_workspace_files",  
    "arguments": {  
      "query": "runtime state"  
    }  
  }  
}
```

Runtime 处理：

```
switch (decision.type) {  
  case "tool_call":  
    return executor.execute(decision.toolCall);  
  
  case "final_answer":  
    return decision.finalAnswer;  
}
```

但这还不够。

因为 Runtime 不应该假设：

LLM 一定知道调用什么。

所以后续可以引入：

```
interface DecisionValidator {  
  validate(decision: AgentDecision, state: RuntimeState): ValidationResult;
```

```
}
```

例如：

```
用户要求实时订单数据
|
v
LLM 直接回答
|
v
DecisionValidator 发现没有 tool call
|
v
要求重新决策或触发策略
```

这就是后面 Planning、Guardrail、Policy、Reflection 的基础。

工业级实现 Notes

工业 Agent 中，Tool Decision 通常不是孤立模块。

更完整的结构是：

```
Goal Manager
|
Task Planner
|
Decision Layer
|
Policy Engine
|
Tool Runtime
```

简单 Agent：

```
LLM -> Tool
```

生产 Agent：

```
LLM -> Decision -> Policy -> Execution
```

在企业场景中，Tool Decision 常常和 Workflow 结合：

```
用户输入
|
v
Intent Classification
|
v
Workflow Selection
|
v
Tool Execution
```

例如客服退款：

```
用户：我要退款
|
v
Intent: Refund Request
|
v
Workflow:
- 检查订单
- 检查退款条件
- 创建退款单
|
```

```
v
Tools:
- query_order()
- create_refund()
```

这说明企业 Agent 并不总是把全部决策权交给 LLM。

更常见的是：

```
业务流程提供稳定边界
LLM 提供理解、路由和异常处理能力
Runtime 提供执行控制
```

工业术语映射

| 本课程概念 | 工业术语 | 说明 || --- | --- | --- || Tool Decision | Tool Selection / Action Selection | 模型选择下一步行动 || Goal-driven Action Selection | Goal-conditioned Decision Making | 在目标约束下选择行动 || Tool Definition | Tool Schema / Function Schema | 描述 Runtime 可用能力 || Tool Choice | Tool Calling Mode | 控制 auto / required / none || Runtime Policy | Policy Engine / Guardrail | 验证、阻止或要求确认 || Tool Call Intent | Action Intent / Function Call | LLM 提出的结构化行动意图 || Tool Executor | Action Executor | Runtime 中真正执行工具的模块 || Observation | Tool Result | Tool 执行后的回流结果 || Runtime Control | Control Plane | Runtime 对行动权的控制层 |

面试视角

Q1：LLM 如何决定是否调用 Tool？

回答：

Runtime 会把 Tool Definition 作为上下文提供给模型，LLM 根据当前任务、Runtime State、历史上下文和工具描述生成下一步行动决策。这个决策可能是直接回答，也可能是结构化 Tool Call Intent，之后由 Runtime 验证并执行。

Q2：为什么说 Tool Calling 不是简单函数调用？

回答：

Tool Calling 前面有一个 Decision Layer。LLM 不是被代码中的 if/else 直接分发到某个函数，而是在 Runtime 提供的受控行动空间中，根据 Goal 和 Context 选择下一步 Action。

Q3：Tool 存在为什么不代表 Agent 一定会调用？

回答：

Tool 只是扩大了模型的 action space。模型仍然需要判断当前任务是否需要外部信息、外部动作或验证。如果已有知识足够，合理决策可能是直接回答。如果模型能力不足、Tool Schema 模糊或 Context 不完整，也可能出现该调用却没有调用的情况。

Q4：Agent 和传统 Workflow 最大区别是什么？

回答：

Workflow 的下一步通常由预定义规则决定，而 Agent 的下一步由 LLM 根据 Context 和 Available Tools 动态选择。但生产 Agent 往往会结合二者：LLM 提出行动建议，Workflow 和 Runtime Policy 约束执行边界。

Q5：企业 Agent 为什么不会完全把 Tool 决策权交给 LLM？

回答：

因为 Tool Decision 涉及权限、成本、安全、合规和业务状态。LLM 可以提出行动意图，但 Runtime 必须通过 Policy、Guardrail、Human Approval 和 Workflow Constraint 决定是否允许执行。

Q6：Claude Code 这类 Agent 为什么不只是模型强？

回答：

Claude Code 的能力来自模型、工程化工具、清晰的 Tool Schema、Context Engineering 和 Runtime Loop 的组合。工具边界越清晰，模型越容易做出稳定的 Tool Decision。

写书素材

适合放进书里的英文总结：

Tool Calling is not about giving an LLM access to functions. It is about giving an Agent Runtime a controlled action space where the model can propose the next step and the runtime can decide how that step is executed.

中文版本：

Tool Calling 的本质不是让模型拥有函数调用能力，而是 Runtime 为 Agent 建立一个受控行动空间，让模型提出下一步行动，让 Runtime 负责验证、约束和执行。

本 Part 可以使用的章节标题：

Tool Decision: From Goal to Action

或者：

LLM Tool Decision Is Goal-driven Action Selection

本 Part 核心认知升级

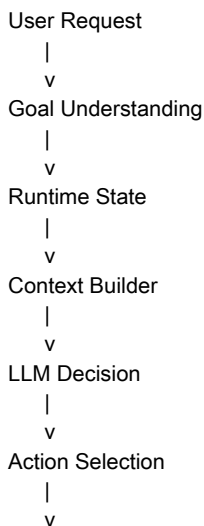
完成 Part B 后，认知应从：

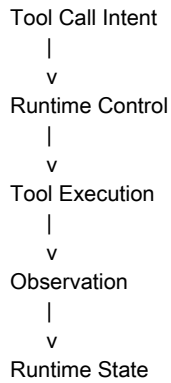
LLM 会不会调用 Tool？

升级为：

Agent Runtime 如何让 LLM 在目标约束下选择下一步行动？

完整链路：





Day04 的核心是：

Runtime 如何让 LLM 思考

Day05 的核心是：

Runtime 如何让 LLM 行动

Part B 是行动系统的第一步：

Goal -> Action

下一节学习计划

Day05 Part C : Tool Schema Design。

下一节进入：

如何设计一个让 LLM 正确理解和调用的 Tool？

重点回答：

为什么 Tool Schema 是 Agent 的 API Contract；

name / description / parameter 如何设计；

为什么一个万能 Tool 是坏设计；

为什么 Claude Code 要拆分大量小 Tool；

Tool Schema 如何影响 Agent 行为；

Tool Schema 如何决定 Agent 能力边界；

如何设计工业级 Tool。

本章思考题

为什么 Tool 存在不代表 Agent 一定会调用？

Tool Decision 和普通函数调用有什么区别？

为什么 Goal Understanding 是 Tool Decision 的前置？

Tool Schema 为什么会影响模型的 Tool Selection？

Tool Decision 为什么不是 if/else Rule？

Runtime Policy 在 Tool Decision 中有什么作用？

为什么企业 Agent 通常不会完全把 Tool 决策权交给 LLM？

如果模型直接回答了一个需要实时数据的问题，Runtime 可以如何处理？

为什么 Tool 越多不一定让 Agent 越强？

如何用一句话解释 Tool Decision 与 Execution Layer 的分离？

Source

- ChatGPT 分享学习记录：<https://chatgpt.com/share/6a62d9f9-379c-83e8-bd79-ad9b33dbf3a5>

- 本地源记录：[source/day05-part-b-chatgpt-share-source.md](#)